UNIVERSIDAD DON BOSCO

Desarrollo de Software para Móviles - DSM941



PROYECTO DE CÁTEDRA

Etapa 3, Foro 1: Avance Android con Jetpack Compose

Jetpack Compose aplicado al proyecto Pet Home: migración de la lógica en consola a una aplicación Android para el personal del Centro Médico Veterinario Pet Home

Nombre del equipo: **PremierCoders**
ID del Equipo: **T-036**

Integrantes del grupo

Nombre completo	Carnet
Diego Carlos Nochez Gallardo	NG182027
Nelson Elías Henríquez Chávez	HC252054
Jimmy Royer Melendez Pineda	MP251278

San Salvador, 18 de octubre de 2026

Jetpack Compose aplicado al proyecto Pet Home

Pet Home, Centro Médico Veterinario, es una clínica de San Salvador que ya usa VetPraxis para su agenda y expedientes, pero que lleva fuera de ese sistema tres procesos críticos: el cierre de caja y la emisión de comprobantes de crédito fiscal en hojas de Excel, el envío de fotografías clínicas por el WhatsApp personal de la veterinaria y el seguimiento de los pacientes después de un tratamiento. En la Etapa 2 se construyó en Kotlin, en modo consola, el núcleo de una herramienta interna para el personal que resuelve esos tres problemas. Este documento explica cómo esa lógica se migró a una aplicación Android con Jetpack Compose en el Foro 1 de la Etapa 3, qué conceptos de Compose se usaron y por qué cada decisión de diseño responde tanto a las características del framework como a las necesidades de la clínica.

El documento se organiza en cuatro partes. Primero se resumen los fundamentos de Jetpack Compose que sostienen la implementación. Después se describe la arquitectura de la aplicación y se justifica cada decisión técnica. Luego se presentan las pantallas, con capturas de la aplicación en ejecución, y las pruebas realizadas. Por último se exponen las dificultades encontradas, los pasos para el Foro 2 y las conclusiones.

Fundamentos de Jetpack Compose

Interfaz declarativa

Jetpack Compose es el kit de herramientas moderno de Android para construir interfaces de forma declarativa. En el modelo tradicional con vistas XML, el programador obtiene referencias a cada vista y las modifica manualmente cuando cambian los datos (por ejemplo, `textView.setText(...)`), lo que obliga a mantener sincronizados a mano el estado y la pantalla. En Compose, en cambio, la interfaz se describe como una función de los datos: el programador

declara cómo debe verse la pantalla para un estado determinado y el framework se encarga de actualizarla cuando ese estado cambia (Android Developers, s.f.-f).

Para Pet Home esta diferencia es central. Varias pantallas deben reaccionar a cada tecla que escribe el usuario: el total esperado de un cierre de caja, el IVA de un crédito fiscal o la fecha de control de un seguimiento. Con vistas XML esto habría requerido escuchadores de texto y actualizaciones manuales en cada campo; con Compose basta con calcular el valor a partir del estado actual y mostrarlo.

Funciones composables y recomposición

La unidad básica de Compose es la función anotada con `@Composable`, que emite elementos de interfaz en lugar de devolver un valor. Cuando un dato que una función composable lee cambia, Compose vuelve a ejecutar únicamente las funciones afectadas; a este proceso se le llama **recomposición**. Las funciones composables deben ser rápidas, no depender del orden de ejecución y no producir efectos secundarios, porque pueden ejecutarse muchas veces (Android Developers, s.f.-f). Por esa razón, en Pet Home las acciones que cambian datos (registrar un cierre, anular un crédito) nunca se ejecutan durante el dibujo, sino dentro de los manejadores `onClick`, y los efectos que deben ocurrir una sola vez, como redirigir al login cuando no hay sesión, se encapsulan en `LaunchedEffect`.

Estado y flujo unidireccional de datos

En Compose el estado es cualquier valor que puede cambiar con el tiempo y que la interfaz debe reflejar. La función `mutableStateOf` crea un estado observable; `remember` lo conserva entre recomposiciones y `rememberSaveable` además lo conserva cuando la actividad se recrea, por ejemplo al rotar el teléfono (Android Developers, s.f.-e). La documentación oficial recomienda la **elevación de estado** (*state hoisting*): los componentes reciben el valor que

muestran y una función para notificar cambios, en lugar de guardar el estado internamente. Así se obtiene un flujo unidireccional en el que el estado baja hacia la interfaz y los eventos suben hacia quien lo administra (Android Developers, s.f.-e).

Para el estado que pertenece a la lógica de negocio y no a una pantalla concreta se usa un `ViewModel`, una clase de Android Jetpack que sobrevive a los cambios de configuración y separa los datos de la interfaz (Android Developers, s.f.-g; Künneth, 2023).

Material Design 3 y temas

Compose incluye una implementación de Material Design 3 con componentes listos para usar, como `Scaffold`, `TopAppBar`, `Card`, `OutlinedTextField`, `FilterChip` y `AlertDialog`, y un sistema de temas basado en un esquema de colores, tipografía y formas que se aplica a toda la aplicación desde `MaterialTheme` (Android Developers, s.f.-b).

Navegación

La biblioteca Navigation Compose define un `NavHost` con un grafo de destinos identificados por rutas y un `NavController` que administra la pila de navegación (*back stack*). Cada destino es una función composable, por lo que toda la aplicación puede funcionar con una sola actividad (Android Developers, s.f.-c).

Arquitectura de la aplicación

Del MVC en consola al MVC con Compose

El curso exige como mínimo una arquitectura MVC, y la Etapa 2 ya estaba organizada así. La decisión fue **conservar las capas y reemplazar solo la vista**. El modelo y los gestores de la consola se reutilizaron casi sin cambios, y la consola se sustituyó por funciones composables. Entre el controlador y la vista se agregó un `PetHomeViewModel` que expone el estado observable

que Compose necesita. Esta separación también prepara el Foro 2: cuando las listas en memoria se reemplacen por Firebase, solo cambiarán los gestores y las pantallas quedarán intactas.

Tabla 1

Correspondencia entre las capas MVC y el código del proyecto

Capa	Paquete / clases	Responsabilidad
Modelo	`modelo`: Usuario, Administrador, Recepcionista, CierreCaja, CreditoFiscal, EvidenciaClinica, Seguimiento	Datos del dominio y cálculos propios (IVA, diferencia de caja, fecha de control, vencimiento).
Controlador	`controlador`: GestorCierreCaja, GestorCreditoFiscal, GestorEvidenciaClinica, GestorSeguimiento, GestorReportes, GestorUsuarios	Reglas de negocio y operaciones CRUD, heredadas de la Etapa 2.
Controlador (puente)	`PetHomeViewModel`	Expone el estado observable a Compose y sobrevive a la rotación.
Vista	`vista`: tema, componentes y 8 pantallas; `navegacion`	Interfaz declarativa en Jetpack Compose y grafo de navegación.

Nota. Elaboración propia a partir de la estructura del repositorio PetHome (carpeta PetHomeAndroid).

Decisiones de diseño y su justificación

Una sola actividad y un ViewModel compartido

`MainActivity` solo aplica el tema y carga el grafo de navegación. Todas las pantallas comparten una misma instancia de `PetHomeViewModel`, de modo que un cierre registrado en el módulo de caja aparece de inmediato en el panel y en los reportes. Como el `ViewModel` sobrevive a la rotación (Android Developers, s.f.-g), el personal no pierde lo registrado si gira el teléfono a mitad de un cierre.

Estado inmutable para que Compose detecte los cambios

En la consola, anular un crédito o completar un seguimiento modificaba el objeto directamente (`credito.anulado = true`). Al migrar se detectó que eso no basta en Compose: el

estado observable compara la lista nueva con la anterior y, si los objetos son los mismos, no recompone la pantalla. Por eso las *data classes* del modelo se volvieron inmutables (`val`) y los gestores reemplazan el elemento por una copia con `copy(...)`. El cambio responde directamente a cómo funciona el estado en Compose (Android Developers, s.f.-e) y además hace el código más predecible.

Figura 1

Anulación de un crédito fiscal mediante una copia inmutable

```
fun anular(numero: String): Boolean {
    val index = creditos.indexOfFirst { it.numeroCorrelativo == numero }
    if (index == -1) { LoggerErrores.registrar("..."); return false }
    creditos[index] = creditos[index].copy(anulado = true)
    return true
}
```

Nota. Fragmento de `GestorCreditoFiscal.kt`. El crédito no se elimina, se anula y conserva su número correlativo, como corresponde a un comprobante fiscal.

Cálculos en vivo como consecuencia de la recomposición

En las pantallas de Cierre de Caja y Créditos Fiscales los resultados se muestran mientras el usuario escribe. No hay código especial para lograrlo: en cada recomposición se construye un objeto temporal del modelo con los valores leídos y se reutilizan sus propiedades calculadas. Así la vista no duplica la lógica del modelo, la recepcionista ve el faltante o el IVA antes de confirmar y se reducen los errores que hoy ocurren en Excel.

Figura 2

Vista previa del cierre de caja reutilizando la lógica del modelo

```
val vistaPrevia = CierreCaja(
    id = 0, fecha = LocalDate.now(),
    efectivo = Validador.aNumero(efectivo) ?: 0.0,
    tarjeta = Validador.aNumero(tarjeta) ?: 0.0,
    transferencia = Validador.aNumero(transferencia) ?: 0.0,
    montoContado = Validador.aNumero(contado) ?: 0.0,
    responsable = ""
)
FilaValor("Total esperado", Formato.dinero(vistaPrevia.totalEsperado))
FilaValor("Resultado", vistaPrevia.estadoDiferencia(),
          color = colorDiferencia(vistaPrevia))
```

Nota. Fragmento de `PantallaCierreCaja.kt`. Cada vez que cambia un campo, Compose recompone y recalcula los valores.

Validación de formularios con mensajes por campo

En la consola, el `Validador` leía la entrada y repetía la pregunta hasta obtener un valor válido. Ese modelo no encaja con una interfaz gráfica, donde el usuario puede corregir cualquier campo en cualquier orden. Por eso se rediseñó: cada función recibe el texto y devuelve el mensaje de error o `null`. La vista guarda esos mensajes en el estado y los pasa al componente `CampoTexto`, que activa `isError` y muestra el texto en `supportingText` del `OutlinedTextField` de Material 3. Se validan formato de correo, fortaleza y confirmación de contraseña, montos (no negativos, máximo dos decimales y punto o coma decimal), fechas (formato y que no sean futuras) y rangos enteros.

Figura 3

Validación de montos que devuelve el mensaje de error

```
fun monto(valor: String, campo: String, permitirCero: Boolean = true): String? {
    if (valor.isBlank()) return "$campo es obligatorio"
    val numero = aNumero(valor) ?: return "Ingresa un número válido (ej. 25.50)"
    val decimales = valor.trim().replace(',', '.').substringAfter('.', "")
    return when {
        numero < 0 -> "$campo no puede ser negativo"
        !permitirCero && numero == 0.0 -> "$campo debe ser mayor a 0"
        decimales.length > 2 -> "Usa máximo 2 decimales"
        else -> null
    }
}
```

Nota. Fragmento de `Validador.kt`.

Componentes reutilizables

Para mantener la coherencia visual con los mockups de la Etapa 1 y evitar código repetido se crearon componentes propios: `PantallaBase` (estructura con barra superior teal, botón de volver y avisos tipo *Snackbar*), `CampoTexto` (campo con ícono, validación y botón para mostrar u ocultar la contraseña), `TarjetaDato`, `FilaValor`, `EstadoVacio` y `DialogoConfirmacion`. Siguen el principio de elevación de estado: no guardan datos propios, reciben el valor y una función de cambio, y por eso funcionan igual en las ocho pantallas.

Tema propio y desactivación del color dinámico

Desde Android 12, Material 3 puede tomar los colores del fondo de pantalla del usuario. Para una herramienta institucional se prefirió una identidad estable, así que `PetHomeTheme` define un `lightColorScheme` con la paleta teal de los mockups (#0D7C70 y #129C8C) y colores semánticos para el resultado del cuadre: verde para caja cuadrada, rojo para faltante y ámbar para sobrante (Android Developers, s.f.-b).

Autenticación y permisos por rol

El requisito de dos roles con permisos diferenciados se resolvió con polimorfismo. La clase abstracta `Usuario` declara `puedeAcceder(modulo)`: `Administrador` devuelve verdadero

para los cinco módulos y `Recepcionista` solo para Cierre de Caja y Créditos Fiscales, que son las tareas de recepción descritas en la entrevista. El panel construye sus tarjetas con `usuario.modulosPermitidos()`, así que cada rol ve únicamente lo suyo. Además, cada destino del grafo pasa por la función `ConSesion`, que redirige al login si no hay sesión y muestra «Acceso restringido» si el rol no tiene permiso, lo que protege la navegación aunque alguien intente llegar a una ruta directamente.

Navegación y control de la pila

El grafo define ocho rutas. Al iniciar sesión, registrarse o cerrar sesión se navega con `popUpTo` sobre el grafo completo y `inclusive = true`, de modo que la pila queda vacía (Android Developers, s.f.-c). Gracias a eso, después de cerrar sesión el botón Atrás no regresa al panel, un detalle de seguridad importante en un equipo compartido por el personal de recepción.

Evidencia clínica con el selector de fotos del sistema

Para sustituir el envío de fotos por WhatsApp se usó el selector de fotos de Android (*Photo Picker*) mediante `rememberLauncherForActivityResult` con el contrato `PickVisualMedia`. Este selector no requiere solicitar permisos de almacenamiento, porque el usuario elige exactamente qué imagen comparte (Android Developers, s.f.-d), lo que es adecuado para fotos clínicas sensibles. Las miniaturas se muestran con `AsyncImage` de la biblioteca Coil, que carga imágenes de forma asíncrona y se integra con Compose (Coil Contributors, s.f.).

Pantallas implementadas

La aplicación tiene ocho pantallas: las seis diseñadas en los mockups de la Etapa 1 más Registro y Seguimiento. La Tabla 2 relaciona cada pantalla con los componentes de Compose que utiliza y con el requisito del foro que cubre.

Tabla 2

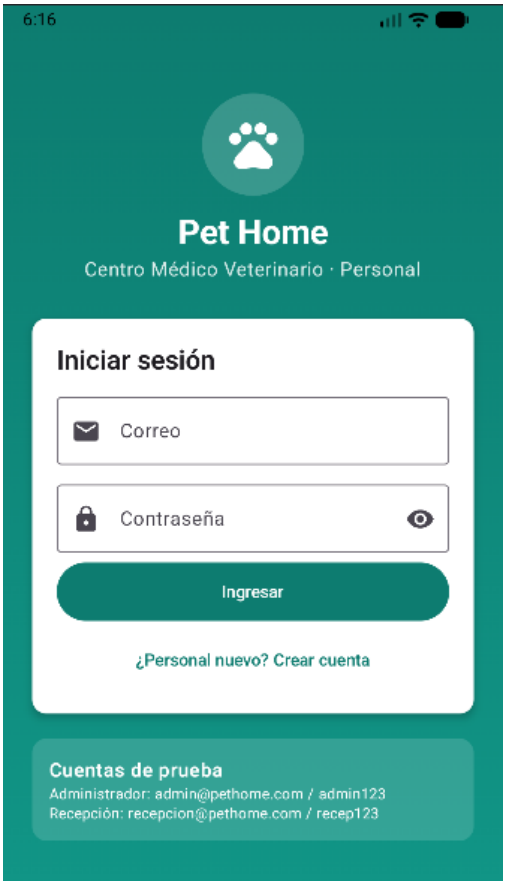
Pantallas, componentes principales y requisitos del Foro 1

Pantalla	Componentes principales	Requisito que cubre
Login	Column con desplazamiento, Card, CampoTexto, Button	1. Autenticación · 4. Validación
Registro	CampoTexto, FilterChip (rol), Button	1. Autenticación · 4. Validación
Panel principal	Scaffold, TarjetaDato, Card con onClick, AlertDialog	2. Módulo principal · 3. Navegación
Cierre de Caja	LazyColumn, CampoTexto, FilterChip, AlertDialog de edición	2. Módulo · 4. Validación
Créditos Fiscales	LazyColumn, AssistChip (correlativo), FilterChip, OutlinedButton	2. Módulo · 4. Validación
Evidencia Clínica	LazyColumn, LazyRow, AsyncImage, Dialog, Photo Picker	2. Módulo · 4. Validación
Seguimiento	LazyColumn, FilterChip con desplazamiento horizontal	2. Módulo · 4. Validación
Reportes	Card, LinearProgressIndicator, TarjetaDato	2. Módulo

Nota. Todas las pantallas comparten `PantallaBase` y el tema `PetHomeTheme`.

Figura 4

Pantalla de inicio de sesión ejecutándose en el emulador



Nota. Captura propia en el emulador Medium Phone API 37. El encabezado conserva el degradado teal y la huella de los mockups de la Etapa 1.

Figura 5

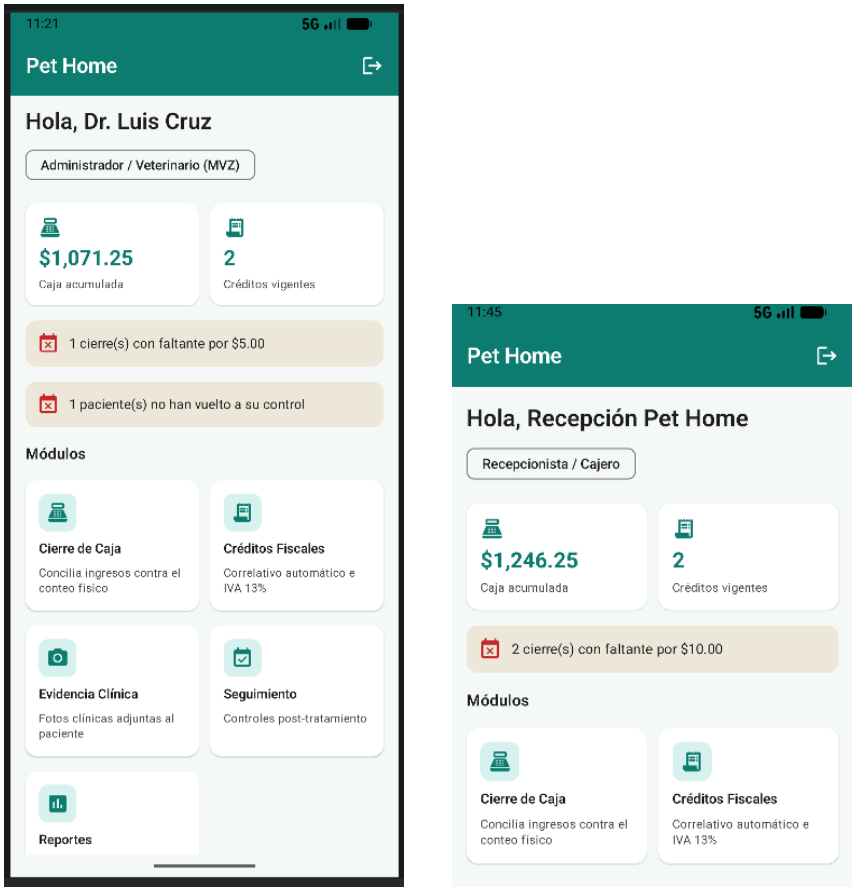
Validación de formularios en el inicio de sesión



Nota. Captura propia. Al presionar Ingresar con los campos vacíos, cada campo muestra su propio mensaje de error mediante `isError` y `supportingText`.

Figura 6

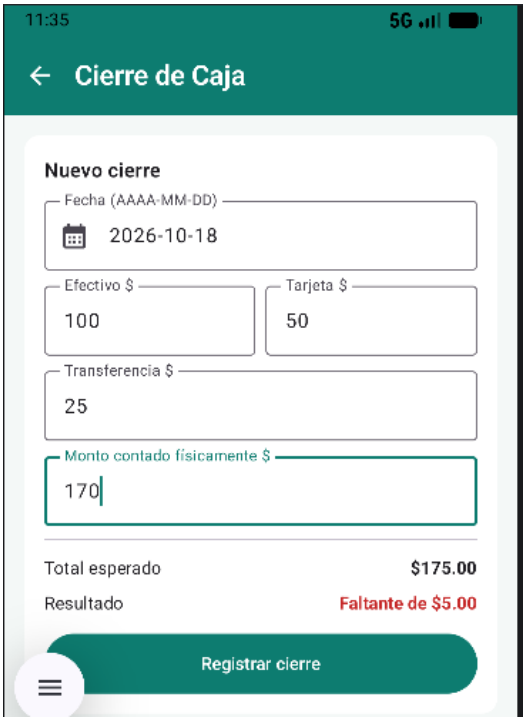
Panel principal según el rol: Administrador (izquierda) y Recepcionista (derecha)



Nota. Captura propia. Los módulos visibles se generan con el método polimórfico `modulosPermitidos()`; la recepcionista solo ve Cierre de Caja y Créditos Fiscales. Los avisos del panel alertan de faltantes de caja y de pacientes que no volvieron a su control.

Figura 7

Cierre de caja con faltante calculado mientras se escribe



Nota. Captura propia. Con $100.00 en efectivo, $50.00 en tarjeta, $25.00 en transferencia y $170.00 contados, la app muestra un total esperado de $175.00 y un faltante de $5.00 antes de registrar el cierre.

Figura 8

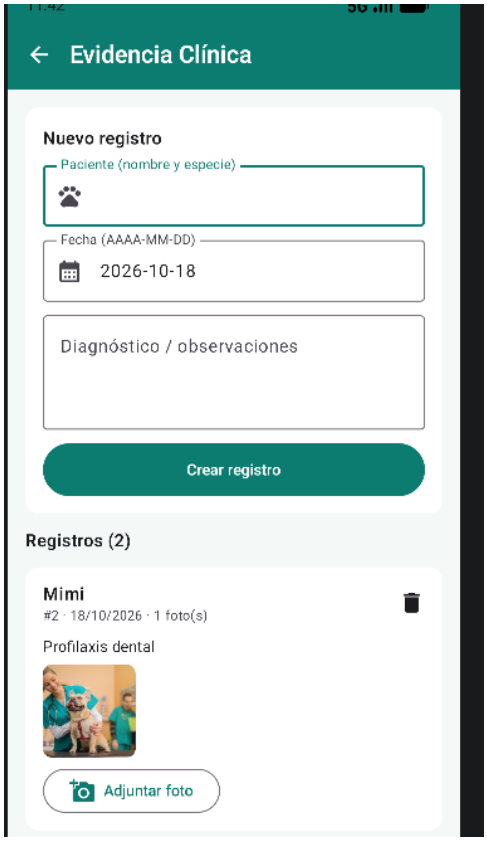
Crédito fiscal emitido con IVA calculado automáticamente



Nota. Captura propia. El correlativo CCF-2026-0003 se asigna automáticamente y el IVA del 13 % se calcula sobre el subtotal, como establece la legislación salvadoreña del IVA (Asamblea Legislativa de la República de El Salvador, 1992).

Figura 9

Evidencia clínica con fotografía adjunta al registro del paciente



Nota. Captura propia. La foto se elige con el selector del sistema, sin pedir permisos de almacenamiento, y queda asociada al registro del paciente en lugar de un chat de WhatsApp. Las imágenes usadas son ilustrativas.

Figura 10

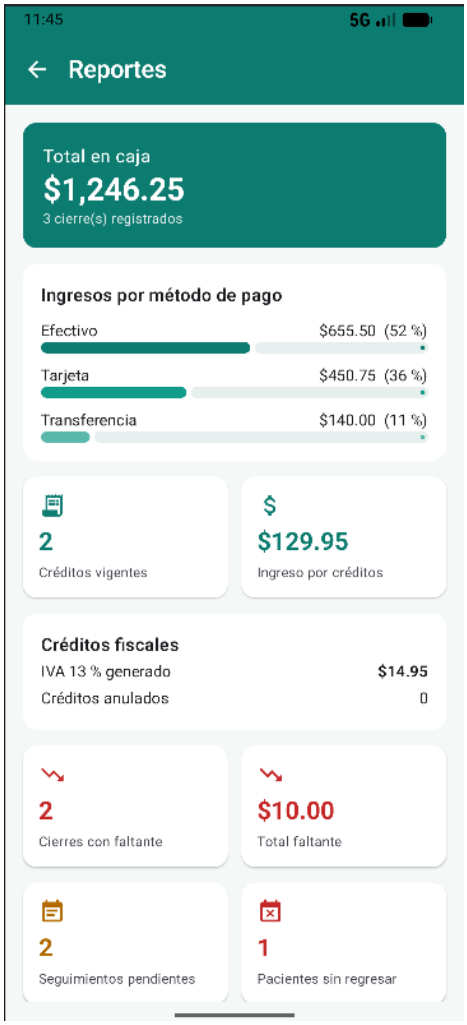
Seguimiento post-tratamiento con paciente vencido



Nota. Captura propia. La fecha de control se calcula al escribir los días, y el paciente que no regresó se marca en rojo; responde al caso real de la entrevista, un paciente de profilaxis dental que no volvió a su control.

Figura 11

Reportes financieros



Nota. Captura propia. Los datos se calculan en `GestorReportes` a partir de los demás módulos; los créditos anulados no se suman a los ingresos.

Pruebas realizadas

La lógica de negocio (modelo, gestores y validador) se compiló de forma independiente con Kotlin 2.0.21 y se verificó con un programa de pruebas automáticas que confirmó los cálculos de IVA, correlativo, faltante y sobrante, fechas de control, seguimientos vencidos y reportes. La interfaz se probó manualmente en el emulador. La Tabla 3 resume los casos principales.

Tabla 3

Casos de prueba principales

Caso	Resultado esperado	Resultado
Login con contraseña incorrecta	Mensaje «Contraseña incorrecta»	Correcto
Registro con correo ya existente	Mensaje «Ya existe una cuenta con ese correo»	Correcto
Monto negativo o con tres decimales	Error debajo del campo; no se registra	Correcto
Fecha futura en un cierre	«La fecha no puede ser futura»	Correcto
Cierre con conteo menor al esperado	«Faltante de $X» en rojo	Correcto
Crédito con subtotal $50.00	IVA $6.50 y total $56.50, correlativo siguiente	Correcto
Anular un crédito	Queda tachado y no suma en reportes	Correcto
Rol Recepcionista	Solo ve Cierre de Caja y Créditos Fiscales	Correcto
Rotar la pantalla con datos escritos	Se conservan datos y campos	Correct
Cerrar sesión y presionar Atrás	No regresa al panel	Correcto

Dificultades y aprendizajes

La primera dificultad fue de entorno. La versión de Android Studio usada (Rabbit, 2026.2.1) trae Java 25, que no es compatible con Gradle 8.11.1 del proyecto; se resolvió indicándole al IDE que usara un JDK 21. Además, el emulador requería activar la plataforma de hipervisor de Windows (WHPX) y reiniciar el equipo. La segunda dificultad fue conceptual: el modelo mutable de la consola no provocaba recomposición, lo que llevó a adoptar estado inmutable. La tercera fue cambiar la forma de validar, de un ciclo de reintentos en consola a mensajes por campo, lo que mostró en la práctica la diferencia entre programación imperativa y declarativa.

Próximos pasos

En el Foro 2 los gestores en memoria se reemplazarán por Firebase Authentication y Cloud Firestore, y las fotos de evidencia se guardarán en un servicio de almacenamiento en la

nube. Gracias a la separación por capas, ese cambio no debería afectar las pantallas. También se agregará una segunda gráfica al módulo de reportes, para cumplir el requisito de al menos dos gráficas del proyecto, y los campos NIT y NRC al crédito fiscal, que un comprobante real exige.

Conclusiones

Jetpack Compose permitió migrar la lógica de la Etapa 2 sin reescribirla: el modelo y los gestores se conservaron y solo se sustituyó la vista. El enfoque declarativo y la recomposición resolvieron de forma natural una necesidad concreta de la clínica, que es ver el IVA, el faltante de caja o la fecha de control mientras se escribe. El manejo explícito del estado obligó a mejorar el diseño del modelo hacia objetos inmutables. Los componentes reutilizables y el tema propio mantuvieron la coherencia con los mockups, y Navigation Compose, junto con el polimorfismo de los roles, aseguró que cada miembro del personal acceda solo a lo que le corresponde.

Referencias

Android Developers. (s.f.-a). *Get started with Jetpack Compose*. Recuperado el 18 de octubre de 2026, de https://developer.android.com/develop/ui/compose/documentation

Android Developers. (s.f.-b). *Material Design 3 in Compose*. Recuperado el 18 de octubre de 2026, de https://developer.android.com/jetpack/compose/themes/material3

Android Developers. (s.f.-c). *Navigation with Compose*. Recuperado el 18 de octubre de 2026, de https://developer.android.com/jetpack/compose/navigation

Android Developers. (s.f.-d). *Photo picker*. Recuperado el 18 de octubre de 2026, de https://developer.android.com/training/data-storage/shared/photo-picker

Android Developers. (s.f.-e). *State and Jetpack Compose*. Recuperado el 18 de octubre de 2026, de https://developer.android.com/develop/ui/compose/state

Android Developers. (s.f.-f). *Thinking in Compose*. Recuperado el 18 de octubre de 2026, de https://developer.android.com/develop/ui/compose/mental-model

Android Developers. (s.f.-g). *ViewModel overview*. Recuperado el 18 de octubre de 2026, de https://developer.android.com/topic/libraries/architecture/viewmodel

Asamblea Legislativa de la República de El Salvador. (1992). *Ley de Impuesto a la Transferencia de Bienes Muebles y a la Prestación de Servicios* (Decreto Legislativo n.º 296).

Coil Contributors. (s.f.). *Coil: Image loading for Android and Compose Multiplatform*. Recuperado el 18 de octubre de 2026, de https://coil-kt.github.io/coil/

Künneth, T. (2023). *Android UI development with Jetpack Compose* (2.ª ed.). Packt Publishing.